

BOOK 16 — MICROSERVICES ARCHITECTURE

Spring Cloud, Service Discovery, API Gateway, Resilience & Distributed Patterns (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 16 of 24 (+1 Special: Book 15A)
Versions Covered: Spring Boot 3.x, Spring Cloud 2023.x (Leyden release train), Resilience4j, Netflix Eureka, Docker, Kubernetes (fundamentals) **Prerequisites:** Book 12 (Spring Boot REST APIs), Book 13 (Spring Data JPA), Book 14 (Spring Security/JWT), Book 15 (Testing) **Next Book:** 17_Kafka_Event_Driven_Architecture.md

★★★ **RECRUITER-PRIORITY NOTE:** Recruiter search data for "Java Full Stack Developer" (3–5 yrs, 6–9 LPA) ranks **Microservices** among the top 5 most-searched keywords, directly behind Spring Boot and Java. This book is built with **special depth** on the exact chain the market is hiring for: **Microservices → API Gateway → Service Discovery → Config Server → Resilience Patterns → Docker → Kubernetes basics** — not a generic "what are microservices" overview.

📖 How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: ఇప్పటివరకు మనం build చేసింది — Spring Boot (Book 12), JPA (Book 13), Security (Book 14) — అన్నీ ఒక **single deployable application** context లో. ఈ పుస్తకం లో అదే knowledge ని **multiple independent services** గా ఎలా విడగొట్టాలో, అవి ఒకదానితో ఒకటి ఎలా communicate అవుతాయో, discover అవుతాయో, fail అయినప్పుడు ఎలా resilient గా ఉండాలో నేర్చుకుంటాము. ఇది recruiter market లో అత్యధికంగా అడిగే skill.

English: Everything built so far — Spring Boot (Book 12), JPA (Book 13), Security (Book 14) — lived inside a **single deployable application**. This book teaches how to split that same knowledge into **multiple independent services**, how they communicate, discover each other, and stay resilient when one of them fails. This is the single most recruiter-demanded skill in the current Java Full Stack market.

🎯 Learning Objectives

1. Understand why and when to move from monolith to microservices (and when NOT to).
2. Decompose a system into services using bounded contexts.
3. Implement centralized configuration with Spring Cloud Config Server.
4. Implement service discovery with Netflix Eureka.
5. Build an API Gateway with Spring Cloud Gateway (routing, filters, rate limiting).
6. Implement resilience patterns (Circuit Breaker, Retry, Bulkhead, Timeout) with Resilience4j.
7. Understand and implement the SAGA pattern for distributed transactions.
8. Understand CQRS and Event Sourcing.

9. Implement distributed tracing and centralized logging.
10. Containerize microservices with Docker and understand Kubernetes fundamentals.
11. Design a complete, production-style microservices architecture.

Table of Contents

Ch.	Title
1	Monolith vs Microservices: Why, When, Trade-offs
2	Service Decomposition & Bounded Contexts
3	Inter-Service Communication: Sync (REST/ RestClient) vs Async
4	★ Spring Cloud Config Server — Centralized Configuration
5	★★★★ Service Discovery — Netflix Eureka
6	★★★★ API Gateway — Spring Cloud Gateway
7	★★★★ Resilience Patterns — Circuit Breaker, Retry, Bulkhead, Timeout (Resilience4j)
8	Distributed Transactions — The SAGA Pattern
9	CQRS (Command Query Responsibility Segregation)
10	Event Sourcing
11	Distributed Tracing & Centralized Logging
12	Docker for Microservices
13	Kubernetes Fundamentals for Java Developers
14	Mini Project — Order + Payment + Notification Microservices System
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — Monolith vs Microservices: Why, When, Trade-offs

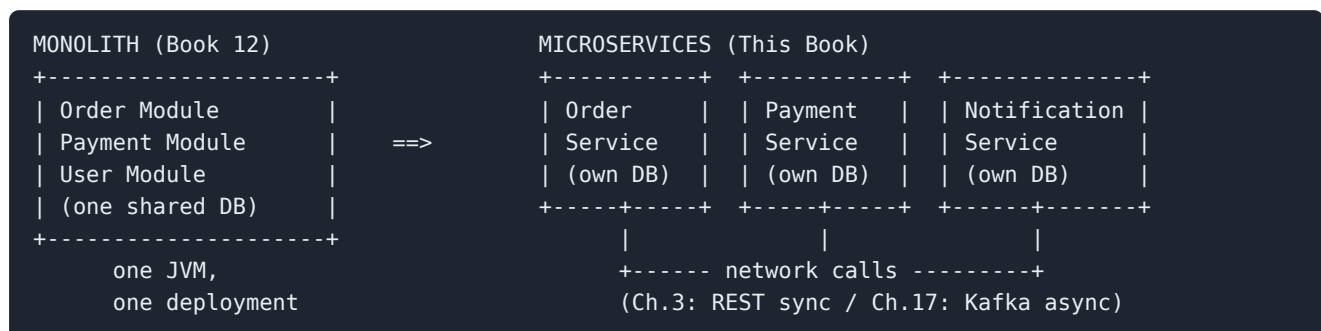
Telugu Explanation

Book 12లో మనం build చేసిన Spring Boot app ఒక **monolith** — ఒకే JVM process లో అన్ని modules (Order, Payment, User) కలిసి run అవుతాయి, ఒకే database share చేస్తాయి. Microservices architecture లో ప్రతి business capability ఒక **independently deployable service** గా విడిపోతుంది — దాని సొంత database, సొంత deployment lifecycle తో. ఇది automatic గా "better" కాదు — ఇది distributed system complexity (network calls, partial failures, data consistency) తీసుకువస్తుంది, బదులుగా independent scaling, independent deployment, team autonomy ఇస్తుంది.

Professional English Explanation

The Spring Boot app we built in Book 12 is a **monolith** — all modules (Order, Payment, User) run in one JVM process and share one database. In microservices architecture, each business capability becomes an **independently deployable service** with its own database and its own deployment lifecycle. This is not automatically "better" — it introduces distributed-system complexity (network calls, partial failures, data consistency across services) in exchange for independent scaling, independent deployment, and team autonomy.

Diagram — Monolith vs Microservices



Internal Working

- **When to use microservices:** the team is large enough that independent deployability matters, different modules have very different scaling needs (Payment needs 10x more capacity than Notification during a sale), or different modules need different technology/release cadence.
- **When NOT to use microservices:** small team, unclear domain boundaries, early-stage product — a "distributed monolith" (many services that must always deploy together) is strictly worse than a monolith; it has all the network complexity with none of the independence benefit.
- **The core trade-off:** a monolith's in-process method call (Book 11's DI-wired object graph) becomes a **network call** in microservices — which can fail, time out, or be slow (Ch.7's resilience patterns exist specifically because of this).

Real-World Example

Amazon's original monolith famously split into hundreds of services as team size grew past what one deployable unit and one release process could support — each team (Cart, Checkout, Recommendations) now owns and deploys its service independently.

Interview Answer

"Microservices split a monolith's modules into independently deployable services, each with its own database. The main benefit is independent scaling and deployment; the main cost is distributed-system complexity — network calls replace in-process method calls, so partial failures, latency, and data consistency across services become real engineering problems. Microservices are justified by organizational/scaling needs, not by architecture fashion — a small team with unclear domain boundaries is usually better off with a well-structured monolith."

Cross Questions

- Q: What replaces an in-process method call between Order and Payment in a microservices world? → A: A network call (Ch.3) — synchronous REST or asynchronous messaging (Book 17) — which introduces latency and failure modes an in-process call never had.
- Q: Why is a "distributed monolith" worse than a monolith? → A: It has multiple deployable services (network complexity) but they still must be deployed together in lockstep (no independence benefit) — the worst of both worlds.

Tricky Questions

- Q: Does using Spring Boot automatically mean you have microservices? → A: No — Spring Boot (Book 12) is just a framework for building one deployable JVM application; whether that application is "a microservice" is an architectural decision about service boundaries and independent deployability, not a framework feature.

Coding Exercise

L1: List three modules in a hypothetical e-commerce app and decide which are good microservice candidates. **L2:** Identify one shared-database dependency that would block splitting two modules apart. **L3:** Draw the monolith-to-microservices diagram for your own past project. **L4 (Interview):** Explain the monolith vs microservices trade-off in under 90 seconds. **L5 (Senior):** Argue for keeping a specific module as part of the monolith rather than splitting it out. **L6 (Mastery):** Write a one-page decision framework for "should this become a microservice?"

CHAPTER 2 — Service Decomposition & Bounded Contexts

Telugu Explanation

Service boundaries ఎలా draw చేయాలి అనేది Domain-Driven Design (DDD) లోని **Bounded Context** concept మీద ఆధారపడి ఉంటుంది — ప్రతి service ఒక స్పష్టమైన business capability (Order, Payment, Inventory, Notification) ని సొంతం చేసుకుంటుంది, దాని సొంత data model తో. తప్పుగా draw చేసిన boundaries (ఉదా: entity మీద split చేయడం కాకుండా business capability మీద) chatty, tightly-coupled services కి దారితీస్తాయి.

Professional English Explanation

How to draw service boundaries is grounded in Domain-Driven Design's **Bounded Context** concept — each service owns one clear business capability (Order, Payment, Inventory, Notification) with its own data model. Boundaries drawn incorrectly (e.g., splitting along entities rather than business capabilities) lead to chatty, tightly-coupled services that defeat the purpose of splitting at all.

Java Code — Bounded Context Boundaries in Package Structure

```
// order-service module — owns Order, OrderItem; does NOT own Product details
package com.example.orderservice.domain;

public class Order {                                // Book 02 - encapsulation
    private Long id;
    private Long customerId;                        // reference by ID only, not a full
    Customer object
    private List<OrderLine> lines;
    private OrderStatus status;
}

record OrderLine(Long productId, int quantity, BigDecimal priceAtOrderTime) {}
// priceAtOrderTime is copied at order time - Order Service does NOT query
// Product Service for price on every read; this avoids tight coupling.
```

Internal Working

- A well-bounded service references other services **by ID**, not by embedding the other service's full object graph — `Order.customerId` (a `Long`), not `Order.customer` (a full `Customer` object) — this is the microservices analog of Book 13's foreign-key relationships, but enforced across a network boundary rather than a database join.
- Copying data at a point in time (`priceAtOrderTime`) rather than always re-fetching live is a deliberate microservices pattern that trades a small amount of duplication for avoiding a network call and a hard dependency on another service being up.
- Getting bounded contexts wrong shows up as **chattiness**: if displaying one Order requires 5 synchronous calls to 5 other services, boundaries were probably drawn along entities, not capabilities.

Real-World Example

An e-commerce Order Service does not store live product prices — it stores the price *at the time of purchase*, exactly like `priceAtOrderTime` above, because an order's total must never silently change if the product's price changes later.

Interview Answer

"Service boundaries should follow Domain-Driven Design's bounded contexts — each service owns one business capability and its own data, referencing other services by ID rather than embedding their data. Point-in-time data copies (like a price snapshot on an order) are a deliberate pattern to avoid excessive cross-service calls. Boundaries drawn along individual entities instead of business capabilities produce chatty, tightly-coupled services that lose the independence microservices are supposed to provide."

Cross Questions

- Q: Why does `Order` store `customerId` instead of a full `Customer` object? → A: To avoid tight coupling to Customer Service's internal data model and avoid a network call just to read an order; it references Customer Service by ID only.
- Q: What is a symptom of poorly-drawn service boundaries? → A: Excessive synchronous inter-service calls ("chattiness") required to fulfill a single logical operation.

Tricky Questions

- Q: Is copying `priceAtOrderTime` onto the Order "bad data duplication"? → A: No — it is a deliberate, correct pattern in distributed systems; the alternative (always querying Product Service live) couples order history to product-service uptime and would incorrectly change historical order totals if prices changed later.

Coding Exercise

L1: Design bounded contexts for Order, Payment, Inventory, Notification services. **L2:** Identify which fields each service should store by ID vs by value (snapshot). **L3:** Refactor a hypothetical `Order` that wrongly embeds a full `Customer` object. **L4 (Interview):** Explain bounded contexts and why they matter for service boundaries. **L5 (Senior):** Review a service design with 6 synchronous calls per request and propose a fix. **L6 (Mastery):** Design bounded contexts for a ride-sharing app end-to-end.

CHAPTER 3 — Inter-Service Communication: Sync (REST) vs Async

Telugu Explanation

Services ఒకదానితో ఒకటి communicate అవ్వడానికి రెండు ప్రధాన మార్గాలు: **Synchronous** (REST call చేసి response కోసం wait చేయడం — Book 12's `RestClient / WebClient`) మరియు **Asynchronous** (message queue మరియు event పంపి, వెంటనే continue అవ్వడం — Book 17's Kafka). Sync సులభం కానీ caller ని callee availability మరియు couple చేస్తుంది; Async decoupled గా ఉంటుంది కానీ complex గా ఉంటుంది (eventual consistency).

Professional English Explanation

Services communicate in two main ways: **Synchronous** (making a REST call and waiting for the response — Book 12's `RestClient / WebClient`) and **Asynchronous** (publishing an event to a message queue and continuing immediately — Book 17's Kafka). Sync is simpler but couples the caller's availability to the callee's; Async is decoupled but more complex (eventual consistency).

Java Code — Synchronous Inter-Service Call

```
import org.springframework.web.client.RestClient;
import org.springframework.web.client.HttpClientErrorException;

@Service
class OrderService {

    private final RestClient restClient; // Spring Boot 3.2+, replaces
    RestTemplate

    OrderService(RestClient.Builder builder) {
        this.restClient = builder.baseUrl("http://payment-service").build(); // Ch.5: resolved
        via Eureka
    }

    PaymentResult chargeCustomer(Long orderId, BigDecimal amount) {
        try {
            return restClient.post()
                .uri("/api/payments")
                .body(new PaymentRequest(orderId, amount))
                .retrieve()
                .body(PaymentResult.class); // blocks until Payment
        } catch (HttpClientErrorException.NotFound e) {
            throw new PaymentServiceUnavailableException(orderId, e); // Book 04 - custom
        }
    }
}
```

Internal Working

- `RestClient` (the modern replacement for `RestTemplate`) makes a real HTTP call over the network — this is **fundamentally different** from Book 11's DI-wired in-process call: it can time out, the target service can be down, and the network itself can fail — none of which apply to a plain Java method call.
- Every synchronous call is a **coupling point**: if Payment Service is down, `chargeCustomer` fails immediately — this is precisely the failure mode Ch.7's Circuit Breaker exists to manage gracefully.
- The base URL `http://payment-service` is not a literal hostname — it is a **logical service name** resolved at call time by the service registry (Ch.5), not a hardcoded IP/port.

Real-World Example

An e-commerce checkout flow typically uses sync REST for the immediate "charge the card and tell me now if it succeeded" step (Payment Service), but async messaging for "send a confirmation email" (Notification Service) — the customer doesn't need to wait for the email to be sent.

Interview Answer

"Synchronous communication (REST, via `RestClient`) is used when the caller needs an immediate answer — like charging a payment — but it couples the caller's availability to the callee's and can fail on timeout or downtime. Asynchronous communication (message queues, Kafka in Book 17) is used when the caller doesn't need to wait — like sending a notification — decoupling the two services' availability at the cost of eventual, not immediate, consistency. Choosing between them is a per-operation architectural decision, not an all-or-nothing choice for the whole system."

Deep Interview Answer (Senior/Architect)

"The real decision criterion is: does this operation need a synchronous response to decide what happens next? Payment authorization genuinely needs sync (the order flow branches on success/failure). A confirmation email does not — it's fire-and-forget, so async avoids introducing an unnecessary dependency on Notification Service's uptime into the checkout critical path. A common senior-level mistake is making everything synchronous 'for simplicity,' which turns a distributed system into effectively a chain of tightly-coupled dependencies — if any one service in that chain is slow or down, the whole chain fails, which defeats microservices' core promise of independent failure domains."

Cross Questions

- Q: What is fundamentally different about a `RestClient` call vs a Book 11 DI-wired method call?
→ A: It's a real network call — subject to timeouts, network failures, and target-service downtime — none of which a plain in-process method call is exposed to.
- Q: Why use async messaging for a confirmation email instead of a sync call? → A: The caller doesn't need to wait for or react to the email being sent, so a sync call would introduce unnecessary coupling to Notification Service's availability.

Tricky Questions

- Q: If Payment Service is critical, should Order Service call it synchronously or asynchronously? →
A: Synchronously — the order flow needs to know immediately whether payment succeeded to decide the next step (confirm order vs reject); this is the correct use case for sync despite the coupling cost.

Coding Exercise

L1: Write a `RestClient`-based call from Order Service to Payment Service. **L2:** Add a `try/catch` for `HttpClientErrorException` and map it to a custom exception. **L3:** Classify five inter-service operations in a hypothetical system as sync or async. **L4 (Interview):** Explain sync vs async inter-service communication trade-offs. **L5 (Senior):** Identify an over-synchronous system design and propose which calls to make async. **L6 (Mastery):** Design the full sync/async call graph for an order fulfillment system.

CHAPTER 4 — Spring Cloud Config Server: Centralized Configuration

Telugu Explanation

Book 12, Ch.8 లో మనం `application.yml` ని ఒక్క service లో నేర్చుకున్నాము. Microservices లో 10+ services ఉంటే, ప్రతి service కి విడివిడిగా config manage చేయడం కష్టం — **Spring Cloud Config Server** ఒక centralized, git-backed configuration source అందిస్తుంది; ప్రతి service startup లో దాని config ని ఒకే చోట నుండి fetch చేసుకుంటుంది.

Professional English Explanation

Book 12, Ch.8 taught `application.yml` for a single service. With 10+ microservices, managing configuration separately per service becomes unmanageable — **Spring Cloud Config Server** provides a centralized, git-backed configuration source; each service fetches its config from one place at startup.

Java Code — Config Server Setup

```
// config-server module
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}
```

```
# config-server's application.yml
server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/example/config-repo    # centralized config repo
          default-label: main
```

```
# order-service's bootstrap.yml — fetches its own config from Config Server
spring:
  application:
    name: order-service    # matches order-service.yml in config-repo
  config:
    import: "configserver:http://localhost:8888"
```

Internal Working

- On startup, `order-service` calls Config Server at `http://localhost:8888/order-service/default`, which reads `order-service.yml` from the git-backed `config-repo` and returns it as JSON — the service's Spring Environment (Book 11, Ch.3's `ApplicationContext` machinery) is populated from this response, exactly as if it were a local `application.yml`.
- Config Server supports **profile-specific** files (`order-service-prod.yml`), reusing Book 12's profile mechanism, but now centrally version-controlled with a full git history of every config change.
- **Refresh without restart:** combined with Spring Cloud Bus and `@RefreshScope`, a config change in git can be pushed to running services without redeploying — a production capability a plain `application.yml` file baked into a JAR (Book 12, Ch.14) cannot offer.

Real-World Example

A team running 20 microservices keeps all database URLs, feature flags, and rate limits in one git-backed config repo — changing a rate limit for all services touches one file and one commit instead of 20 separate deployments.

Interview Answer

"Spring Cloud Config Server centralizes configuration for many microservices in one git-backed repository. Each service fetches its config at startup via `spring.config.import: configserver:...`, matched by its `spring.application.name`. This solves the operational problem of managing dozens of separate `application.yml` files, and combined with `@RefreshScope`, allows config changes to propagate to running services without a redeploy."

Cross Questions

- Q: How does `order-service` know which config file to fetch from Config Server? → A: By its `spring.application.name` (`order-service`), matched against `order-service.yml` in the config repo.
- Q: What Book 12 mechanism does Config Server's profile support build on? → A: Spring profiles (`application-prod.yml`), now centrally git-versioned instead of per-service local files.

Tricky Questions

- Q: Is Config Server a single point of failure for the whole system? → A: It's a risk if not made highly available — if Config Server is down when a service *starts up*, that service can fail to boot; production setups typically run multiple Config Server instances behind the Ch.5 service registry, and services cache their last-fetched config.

Coding Exercise

L1: Set up a Config Server pointing at a local git repo with one service's config file. **L2:** Configure `order-service` to import its config from Config Server at startup. **L3:** Add a `-prod` profile-specific config file and verify it's picked up. **L4 (Interview):** Explain what problem Config Server solves and

how a service fetches its config. **L5 (Senior):** Design a Config Server high-availability setup. **L6 (Mastery):** Explain `@RefreshScope` and dynamic config reload without a redeploy.

CHAPTER 5 — ★★☆☆ Service Discovery: Netflix Eureka

Telugu Explanation

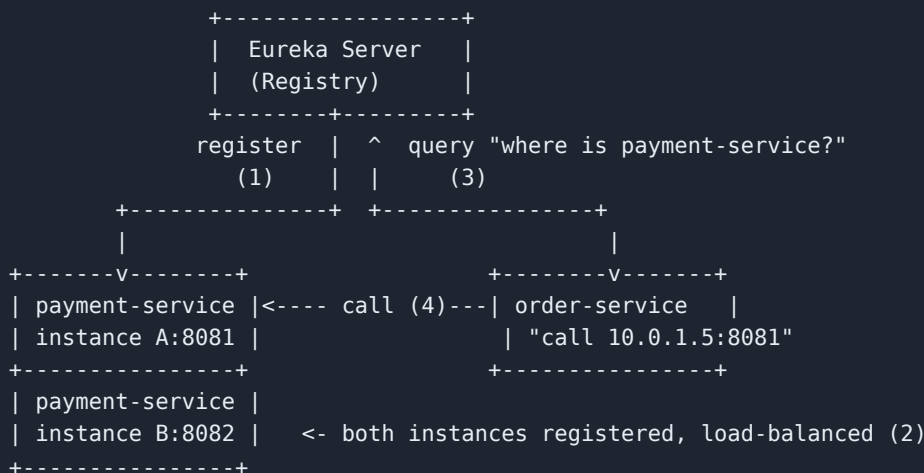
Ch.3 లో `http://payment-service` అని hardcode చేయలేదు అని చెప్పాము — ఎందుకంటే microservices dynamically scale అవుతాయి (multiple instances, changing IPs, especially Docker/Kubernetes లో).

Eureka Server ఒక registry — ప్రతి service startup అయినప్పుడు తనను తాను register చేసుకుంటుంది ("I am payment-service at 10.0.1.5:8080"), మరియు ఇతర services దీన్ని query చేసి actual location కనుక్కుంటాయి. ఇది recruiter-market లో అత్యధికంగా అడిగే core microservices skill.

Professional English Explanation

Chapter 3 avoided hardcoding `http://payment-service` because microservices scale dynamically (multiple instances, changing IPs, especially under Docker/Kubernetes). **Eureka Server** is a registry — each service registers itself on startup ("I am payment-service at 10.0.1.5:8080"), and other services query it to find the actual location. This is one of the most recruiter-demanded core microservices skills in the current market.

Diagram — Service Discovery Flow



Java Code — Eureka Server

```
// eureka-server module
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Java Code — Registering a Client Service and Calling via Logical Name

```
// payment-service's application.yml
// spring.application.name: payment-service
// eureka.client.service-url.defaultZone: http://localhost:8761/eureka

import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
import org.springframework.cloud.client.loadbalancer.LoadBalanced;
import org.springframework.web.client.RestClient;

@SpringBootApplication
@EnableDiscoveryClient // registers this service with
// Eureka on startup
public class PaymentServiceApplication { /* ... */ }

@Configuration
class RestClientConfig {
    @Bean
    @LoadBalanced // resolves "payment-service"
    via Eureka + client-side load balancing
    RestClient.Builder loadBalancedRestClientBuilder() {
        return RestClient.builder();
    }
}
```

Internal Working

- Each Eureka client sends a **heartbeat** to the registry every 30 seconds by default; if the registry misses heartbeats for a configured window (default 90 seconds), it evicts the instance as DOWN — this is how the registry stays accurate as instances crash or scale down.
- `@LoadBalanced` on the `RestClient.Builder` means calling `http://payment-service/...` doesn't hit DNS — Spring Cloud LoadBalancer intercepts the call, asks Eureka for all healthy instances of `payment-service`, and picks one (round-robin by default) — this is **client-side load balancing**, distinct from a traditional server-side load balancer.
- Self-preservation mode:** if Eureka Server sees too many heartbeats missing at once (suggesting a network partition rather than real instance failures), it stops evicting instances to avoid a cascading false-failure scenario — a well-known interview trap for "why didn't Eureka remove a dead instance?"

Real-World Example

Netflix originally built Eureka to handle thousands of dynamically-scaling service instances across AWS availability zones, where hardcoded IPs would be both impossible to maintain and instantly stale.

Interview Answer

"Eureka Server is a service registry — each microservice registers itself on startup and sends periodic heartbeats. Other services discover instances by querying Eureka for a logical service name rather than a hardcoded host/port, and Spring Cloud LoadBalancer performs client-side load

balancing across the returned healthy instances. This solves the problem of dynamic, scaling service instances whose network locations aren't stable."

Deep Interview Answer (Senior/Architect)

"A subtlety senior engineers are expected to know: Eureka favors **availability over strict consistency** (an AP system in CAP terms) — it can return a slightly stale list of instances rather than fail the lookup, and its self-preservation mode deliberately keeps possibly-dead instances registered during suspected network partitions rather than risk evicting healthy ones. This is a deliberate trade-off: for service discovery, a stale-but-available registry (occasionally routing to a since-dead instance, which the client's retry/circuit-breaker in Ch.7 handles) is preferable to an unavailable registry that blocks all service communication."

Cross Questions

- Q: What does `@LoadBalanced` actually change about a `RestClient` call? → A: It resolves the logical service name via Eureka to an actual healthy instance and load-balances across all registered instances, instead of requiring a literal hostname.
- Q: Why might Eureka temporarily keep a dead instance registered? → A: Self-preservation mode — if too many heartbeats are missed at once, Eureka suspects a network partition rather than real failures and pauses eviction to avoid a cascading false-failure.

Tricky Questions

- Q: Is Eureka a CP or AP system in CAP terms, and why does that matter? → A: AP (availability-favoring) — it deliberately tolerates returning stale data over becoming unavailable, which is the correct trade-off for service discovery, where a slightly-stale-but-available registry keeps the system functioning.

Coding Exercise

L1: Set up a Eureka Server and register one client service. **L2:** Register two instances of the same service and observe both in the Eureka dashboard. **L3:** Make a `@LoadBalanced RestClient` call using the logical service name. **L4 (Interview):** Explain client-side load balancing via Eureka. **L5 (Senior):** Explain self-preservation mode and why it's a deliberate trade-off, not a bug. **L6 (Mastery):** Design a multi-zone Eureka deployment for high availability.

CHAPTER 6 — ★★ API Gateway: Spring Cloud Gateway

Telugu Explanation

Client (Book 15A's React app) directly ప్రతి microservice ని call చేస్తే, client కి అన్ని service locations తెలిసి ఉండాలి, మరియు cross-cutting concerns (auth, rate limiting, logging) ప్రతి service లో duplicate అవుతాయి. **API Gateway** ఒకే entry point గా ఉండి, requests ని సరైన service కి route చేస్తుంది, మరియు auth/rate-limiting/logging ని ఒక్క చోట centralize చేస్తుంది. ఇది recruiter-searched core skill.

Professional English Explanation

If the client (Book 15A's React app) called every microservice directly, it would need to know every service's location, and cross-cutting concerns (auth, rate limiting, logging) would be duplicated in every service. An **API Gateway** acts as the single entry point, routes requests to the correct service, and centralizes auth/rate-limiting/logging in one place. This is a core recruiter-searched skill.

Diagram — API Gateway Position

```
React App (Book 15A)
  |
  v
+-----+
| API Gateway (:8080) | <- single entry point, matches Book 15A's API_BASE_URL
| - JWT validation (Ch.14 concepts, centralized)
| - Rate limiting
| - Routing table
+-----+
  |       |       |
  v       v       v
Order    Payment  Notification  (each discovered via Eureka, Ch.5)
:8081    :8082    :8083
```

Java Code — Gateway Routing Configuration

```
import org.springframework.cloud.gateway.route.RouteLocator;
import org.springframework.cloud.gateway.route.builder.RouteLocatorBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
class GatewayRoutesConfig {

    @Bean
    RouteLocator routes(RouteLocatorBuilder builder) {
        return builder.routes()
            .route("order-service", r -> r.path("/api/orders/**")
                .filters(f -> f
                    .addRequestHeader("X-Gateway", "true")
                    .circuitBreaker(c -> c.setName("orderCB") // Ch.7
                        .setFallbackUri("forward:/fallback/orders")))
                .uri("lb://order-service")) // "lb://" = load-
balanced via Eureka
            .route("payment-service", r -> r.path("/api/payments/**")
                .uri("lb://payment-service"))
            .route("notification-service", r -> r.path("/api/notifications/**")
                .uri("lb://notification-service"))
            .build();
    }
}
```

```
# gateway's application.yml – rate limiting with Redis
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-rate-limited
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10 # 10 requests/sec sustained
                redis-rate-limiter.burstCapacity: 20 # allow bursts up to 20
```

Internal Working

- `lb://order-service` tells Spring Cloud Gateway to resolve `order-service` through Eureka (Ch.5) exactly like `@LoadBalanced` did for `RestClient` — the gateway itself is just another Eureka client.
- Gateway **filters** (`addRequestHeader`, `circuitBreaker`, `RequestRateLimiter`) run as a chain per route, conceptually identical to Book 10/Book 14's servlet filter chain, but operating at the routing layer rather than inside any single service.
- Centralizing JWT validation (Book 14) at the gateway means individual services can trust an already-validated request (e.g., via a forwarded, gateway-signed header) — though many

production setups still re-validate the JWT per-service (Ch.9's production practices) as defense-in-depth rather than trusting the network boundary alone.

Real-World Example

Netflix's Zuul (and later Spring Cloud Gateway) sits in front of hundreds of backend services, giving every client a single, stable API surface regardless of how backend services are added, removed, or rescaled.

Interview Answer

"Spring Cloud Gateway is the single entry point for all client requests, routing them to the correct backend microservice resolved via Eureka (`lb://service-name`). It centralizes cross-cutting concerns — rate limiting, circuit breaking, header injection, and often JWT validation — so individual services don't have to duplicate that logic. Its filter chain is conceptually the same pattern as a servlet filter chain (Book 10/14), just applied at the routing layer in front of all services."

Cross Questions

- Q: What does the `lb://` prefix in a Gateway route's URI mean? → A: Resolve this logical service name via the service registry (Eureka, Ch.5) and load-balance across its healthy instances, rather than hitting a fixed host.
- Q: Why centralize rate limiting at the Gateway instead of in each service? → A: To enforce one consistent policy at the single entry point instead of duplicating (and potentially inconsistently implementing) it across every backend service.

Tricky Questions

- Q: If the Gateway validates JWTs, should backend services skip their own validation? → A: Not necessarily — many production systems still re-validate at each service as defense-in-depth, since trusting the network boundary alone is risky if an attacker can reach a backend service directly, bypassing the gateway.

Coding Exercise

L1: Configure a Gateway route forwarding `/api/orders/**` to `order-service` via Eureka. **L2:** Add a `RequestRateLimiter` filter using Redis to one route. **L3:** Add a `circuitBreaker` filter with a fallback URI to one route. **L4 (Interview):** Explain the API Gateway's role and its relationship to Eureka. **L5 (Senior):** Design the Gateway's routing table for a 6-service system. **L6 (Mastery):** Justify whether JWT validation should live at the Gateway, each service, or both.

CHAPTER 7 — ★★☆☆ Resilience Patterns: Circuit Breaker, Retry, Bulkhead, Timeout

Telugu Explanation

Ch.3 లో చెప్పినట్లు, network calls fail అవ్వచ్చు. ఒక downstream service slow అయినా/down అయినా, calling service **cascading failure** కి గురికాకుండా ఉండాలంటే resilience patterns అవసరం: **Circuit Breaker** (repeated failures తర్వాత calls ఆపేయడం), **Retry** (transient failures కోసం మళ్ళీ try చేయడం), **Bulkhead** (failures ని isolate చేయడం), **Timeout** (ఎక్కువసేపు wait చేయకుండా ఉండడం). ఇది Spring Boot + Microservices interview లలో అత్యంత తరచుగా అడిగే topic.

Professional English Explanation

As Ch.3 noted, network calls can fail. To prevent a slow or down downstream service from causing **cascading failure** in the calling service, resilience patterns are essential: **Circuit Breaker** (stop calling after repeated failures), **Retry** (retry transient failures), **Bulkhead** (isolate failures), **Timeout** (don't wait indefinitely). This is one of the most frequently asked topics in Spring Boot + Microservices interviews.

Diagram — Circuit Breaker State Machine

```
CLOSED (normal) --[failure rate > threshold]--> OPEN (fail fast, no calls made)
    ^
    |
    | [wait duration elapses]
    |
    |
    +---[test calls succeed]--- HALF_OPEN (a few trial calls allowed)
                                |
                                | [test calls fail] --> back to OPEN
```

Java Code — Resilience4j Circuit Breaker + Retry + Timeout

```
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import io.github.resilience4j.retry.annotation.Retry;
import io.github.resilience4j.timelimiter.annotation.TimeLimiter;
import java.util.concurrent.CompletableFuture;

@Service
class PaymentClient {

    private final RestClient restClient;

    @CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackPayment")
    @Retry(name = "paymentService")
    @TimeLimiter(name = "paymentService")
    CompletableFuture<PaymentResult> chargeAsync(Long orderId, BigDecimal amount) {
        return CompletableFuture.supplyAsync(() -> // Book 08 - async
execution
            restClient.post().uri("/api/payments")
                .body(new PaymentRequest(orderId, amount))
                .retrieve().body(PaymentResult.class));
    }

    // Fallback signature must match + accept the Throwable
    CompletableFuture<PaymentResult> fallbackPayment(Long orderId, BigDecimal amount, Throwable
t) {
        return CompletableFuture.completedFuture(
            new PaymentResult(orderId, PaymentStatus.PENDING_RETRY)); // graceful
degradation, not a crash
    }
}
```

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        sliding-window-size: 10           # evaluate the last 10 calls
        failure-rate-threshold: 50        # open circuit if >=50% fail
        wait-duration-in-open-state: 10s  # stay OPEN for 10s before trying again
  retry:
    instances:
      paymentService:
        max-attempts: 3
        wait-duration: 500ms
  timelimiter:
    instances:
      paymentService:
        timeout-duration: 2s
```

Internal Working

- The Circuit Breaker wraps the call in a **proxy** (Book 11, Ch.7's AOP proxy mechanism, exactly like `@Transactional`) that tracks a sliding window of recent call outcomes; crossing `failure-rate-threshold` flips the breaker to OPEN, where calls **fail immediately without even attempting the network call** — this is the key mechanism that prevents cascading failure: a slow Payment Service doesn't also make Order Service slow and exhaust its threads.

- `@Retry` and `@CircuitBreaker` stacking order matters: Resilience4j applies them so Retry happens **inside** the Circuit Breaker's evaluation — meaning if all retries for one logical call fail, that counts as **one** failure toward the circuit breaker's threshold, not three.
- The fallback method provides **graceful degradation** — returning a `PENDING_RETRY` status instead of propagating an exception all the way to the client, directly connecting to Book 04's exception-handling philosophy of failing predictably rather than crashing.

Real-World Example

Netflix's Hystrix (predecessor to Resilience4j) was built specifically because one slow dependency (a recommendations service) was causing thread-pool exhaustion and cascading outages across the entire Netflix homepage.

Interview Answer

"Resilience4j's Circuit Breaker wraps a risky call in a proxy that tracks a sliding window of failures; once the failure rate crosses a threshold, it opens and fails fast without attempting the network call at all, preventing a struggling downstream service from cascading failure upstream. Retry handles transient failures by retrying with backoff, Timeout bounds how long a call can hang, and Bulkhead isolates thread pools so one failing dependency can't exhaust threads needed by others. Together they're the standard toolkit for making synchronous inter-service calls (Ch.3) resilient."

Deep Interview Answer (Senior/Architect)

"A senior-level nuance: these patterns must be composed correctly — retrying inside an open circuit breaker is pointless (it'll fail fast every time anyway), and retrying a **non-idempotent** operation (like `chargeCustomer`) without an idempotency key can cause a customer to be double-charged if the first attempt actually succeeded but the response was lost. Production systems pass an idempotency key with payment requests specifically so a safe retry doesn't create a duplicate side effect — this is a detail interviewers use to separate candidates who've memorized the annotations from those who've actually operated payment-adjacent services."

Cross Questions

- Q: What happens to calls when a Circuit Breaker is OPEN? → A: They fail immediately (or hit the fallback) without attempting the actual network call, preventing thread exhaustion and cascading failure.
- Q: Why is retrying a non-idempotent payment call dangerous? → A: If the first attempt actually succeeded but the response was lost, a retry could cause the customer to be charged twice, unless an idempotency key prevents the duplicate.

Tricky Questions

- Q: If Retry and Circuit Breaker are both applied, does each retry attempt count separately toward the circuit breaker's failure threshold? → A: No — Resilience4j applies Retry inside the Circuit Breaker's evaluation, so a fully-retried-and-failed logical call counts as one failure, not three.

Coding Exercise

L1: Add `@CircuitBreaker` with a fallback method to a service client call. **L2:** Configure `sliding-window-size` and `failure-rate-threshold` and trigger the circuit to open. **L3:** Add `@Retry` and observe retry attempts in logs on a simulated failure. **L4 (Interview):** Explain the Circuit Breaker state machine (CLOSED/OPEN/HALF_OPEN). **L5 (Senior):** Identify and fix a non-idempotent retry risk in a payment call. **L6 (Mastery):** Design a full resilience strategy (timeout + retry + circuit breaker + bulkhead) for a payment call chain.

CHAPTER 8 — Distributed Transactions: The SAGA Pattern

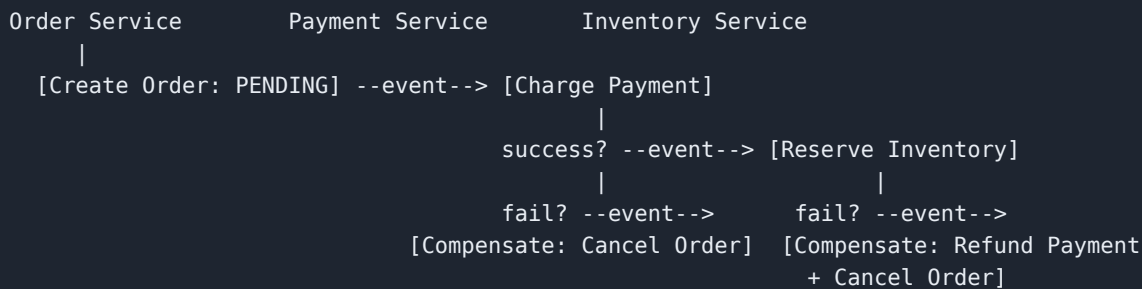
Telugu Explanation

Book 13, Ch.7 లో @Transactional ఒక్క database మీద atomic operations ఇచ్చింది. Microservices లో Order, Payment, Inventory వేర్వేరు databases వాడితే, ఒకే distributed transaction (2PC) practically వాడరు — బదులుగా **SAGA pattern**: ప్రతి step ఒక local transaction, fail అయితే already-completed steps ని **compensating transactions** తో undo చేస్తారు.

Professional English Explanation

Book 13, Ch.7's @Transactional provided atomicity within one database. In microservices, with Order, Payment, and Inventory each using different databases, a single distributed transaction (2PC) is rarely used in practice — instead, the **SAGA pattern**: each step is a local transaction, and if one fails, already-completed steps are undone via **compensating transactions**.

Diagram — SAGA (Choreography Style): Order Placement



Java Code — SAGA Step with Compensating Action

```
@Service
class OrderSagaOrchestrator {

    void placeOrder(OrderRequest request) {
        Order order = orderService.createPending(request); // local TX #1 (Book 13)
        try {
            Ch.3/Ch.7 PaymentResult payment = paymentClient.charge(order.getId(), order.getTotal()); //
            inventoryClient.reserve(order.getId(), request.items()); // local TX #3
            orderService.markConfirmed(order.getId()); // local TX #4
        } catch (PaymentFailedException e) {
            orderService.markCancelled(order.getId()); // compensating action
        } catch (InventoryUnavailableException e) {
            paymentClient.refund(order.getId()); // compensating action
            orderService.markCancelled(order.getId()); // compensating action
        }
    }
}
```

Internal Working

- Each call above (`createPending` , `charge` , `reserve` , `markConfirmed`) is its **own local @Transactional boundary** (Book 13, Ch.7) inside its own service — there is no single database transaction spanning all of them; consistency across services is achieved **eventually**, by running compensating actions if a later step fails.
- **Orchestration** (shown above — one service explicitly directs each step) vs **Choreography** (each service reacts to events from the previous one, no central coordinator) is the key architectural choice: orchestration is easier to understand and debug; choreography is more decoupled but harder to trace end-to-end.
- The compensating transaction for `charge` is `refund` — **not** a database rollback (impossible across services) — it is a deliberate, explicit business operation that undoes the effect of the prior step.

Real-World Example

E-commerce checkout flows almost universally use SAGA: if inventory reservation fails after payment succeeded, the system doesn't "roll back" the payment charge at the database level — it issues an explicit refund transaction.

Interview Answer

"The SAGA pattern replaces a single ACID distributed transaction (impractical across separate microservice databases) with a sequence of local transactions, each in its own service, plus compensating transactions that explicitly undo the effect of prior steps if a later step fails. It comes in two styles: orchestration, where one coordinator explicitly drives each step, and choreography, where each service reacts to events from the previous one with no central coordinator. This achieves eventual consistency across services instead of the immediate consistency Book 13's

`@Transactional` gives within one database."

Cross Questions

- Q: Why can't `@Transactional` from Book 13 span Order Service and Payment Service? → A: Because they use separate databases — `@Transactional` demarcates a transaction against one `DataSource` /connection; it has no mechanism to atomically commit or roll back across two independent databases.
- Q: What is a "compensating transaction"? → A: An explicit business operation (like a refund) that undoes the effect of a previously-completed local transaction, since a true cross-service rollback isn't possible.

Tricky Questions

- Q: Does SAGA give the same consistency guarantee as a single-database ACID transaction? → A: No — it gives **eventual** consistency; there is a window where the system is in an intermediate, not-yet-fully-consistent state (e.g., payment charged but inventory not yet confirmed) before compensation completes.

Coding Exercise

L1: Implement an orchestrated SAGA for order placement with two steps and one compensating action. **L2:** Add a third step and its corresponding compensating action. **L3:** Convert the orchestrated SAGA to a choreography-style design (describe the events, no need to fully implement). **L4 (Interview):** Explain SAGA and why 2PC is avoided in microservices. **L5 (Senior):** Compare orchestration vs choreography trade-offs for a 5-step SAGA. **L6 (Mastery):** Design a full SAGA (with all compensations) for a travel-booking system (flight + hotel + car).

CHAPTER 9 — CQRS (Command Query Responsibility Segregation)

Telugu Explanation

Book 13లో ఒకే `Order` entity reads మరియు writes రెండింటికీ వాడాము. High-scale systems లో read మరియు write patterns చాలా వేరుగా ఉంటాయి (writes అరుదు, reads చాలా ఎక్కువ, differently-shaped). **CQRS** commands (writes) మరియు queries (reads) ని వేర్వేరు models/paths గా విడగొడుతుంది — write model normalized గా ఉండొచ్చు, read model denormalized, dedicated query-optimized గా ఉండొచ్చు.

Professional English Explanation

Book 13 used the same `Order` entity for both reads and writes. In high-scale systems, read and write patterns often differ significantly (writes are rare, reads are frequent and differently-shaped). **CQRS** splits commands (writes) and queries (reads) into separate models/paths — the write model can stay normalized while the read model is denormalized and optimized purely for query performance.

Java Code — Separate Command and Query Models

```
// Command side - writes go through the normalized domain model (Book 13)
@Service
class OrderCommandService {
    @Transactional
    void placeOrder(PlaceOrderCommand cmd) {
        Order order = new Order(cmd.customerId(), cmd.items()); // Book 13's JPA entity
        orderRepository.save(order);
        eventPublisher.publish(new OrderPlacedEvent(order)); // triggers read-model
        update
    }
}

// Query side - reads go through a separate, denormalized, query-optimized model
record OrderSummaryView(Long orderId, String customerName, String status, BigDecimal total) {}

@Service
class OrderQueryService {
    private final OrderSummaryViewRepository viewRepository; // a separate table/store,
    denormalized

    List<OrderSummaryView> getRecentOrdersForCustomer(Long customerId) {
        return viewRepository.findRecentByCustomerId(customerId); // no joins needed -
        already flattened
    }
}

// An event listener keeps the read-model in sync
@Component
class OrderSummaryProjector {
    @EventListener
    void on(OrderPlacedEvent event) {
        viewRepository.save(new OrderSummaryView(
            event.orderId(), event.customerName(), "PLACED", event.total())); // pre-joined,
        flat
    }
}
```

Internal Working

- The command side still uses Book 13's normalized JPA entities and `@Transactional` for correctness on writes; the query side uses a **separately maintained, denormalized** table (or even a different data store entirely, like Elasticsearch) that avoids Book 13, Ch.6's N+1/join costs entirely because it's already pre-joined and flat.
- The two sides are kept in sync via events (`OrderPlacedEvent`) — meaning the read model is **eventually consistent** with the write model, not instantly consistent (the same trade-off Ch.8's SAGA makes, applied to read-performance rather than cross-service transactions).
- CQRS is not "always split reads and writes" — it's justified specifically when read and write models genuinely diverge in shape or scale; applying it to a simple CRUD service adds real complexity for no benefit.

Real-World Example

An analytics dashboard showing "orders per customer, pre-aggregated" is a classic CQRS read model — computing that live from normalized tables on every dashboard view would be far too slow at scale, so a denormalized, pre-computed view is maintained separately.

Interview Answer

"CQRS separates the write model (commands, using Book 13's normalized entities and transactions) from the read model (queries, using a separately maintained, denormalized, query-optimized structure). The two are kept in sync via events, so the read side is eventually consistent with the write side. It's justified when read and write access patterns genuinely diverge — not a default for every service, since it adds real architectural complexity."

Cross Questions

- Q: Why does the query side avoid Book 13's N+1 problem entirely? → A: Because its data is already pre-joined and flattened into a denormalized view, unlike the normalized entity graph the command side uses.
- Q: What keeps the CQRS read model in sync with the write model? → A: Events published on write (e.g., `OrderPlacedEvent`), consumed by a projector that updates the denormalized read model — eventually consistent, not instantly.

Tricky Questions

- Q: Should every microservice use CQRS? → A: No — it's justified only when read/write patterns genuinely diverge in shape or scale; applying it to simple CRUD adds unnecessary complexity and an eventual-consistency window with no real benefit.

Coding Exercise

L1: Design a command model and a separate denormalized query model for Orders. **L2:** Implement an event listener that projects a write into the read model. **L3:** Identify a service in a hypothetical system where CQRS would NOT be justified, and explain why. **L4 (Interview):** Explain CQRS and the consistency trade-off it introduces. **L5 (Senior):** Design the read model for a high-traffic product-catalog search feature. **L6 (Mastery):** Combine CQRS with Ch.10's Event Sourcing for a full audit-friendly order system.

CHAPTER 10 — Event Sourcing

Telugu Explanation

Book 13లో Order state ని ఒకే row గా store చేసాము — ప్రతి update, previous state ని overwrite చేస్తుంది. **Event Sourcing** లో, state ని కాకుండా, ఆ state కి దారితీసిన **events sequence** ని store చేస్తారు (OrderPlaced , PaymentCharged , OrderShipped) — current state ఎప్పుడైనా ఈ events ని replay చేసి derive చేయవచ్చు. ఇది full audit trail ఇస్తుంది.

Professional English Explanation

Book 13 stored Order state as a single row — every update overwrites the previous state. In **Event Sourcing**, instead of state, you store the **sequence of events** that led to that state (OrderPlaced , PaymentCharged , OrderShipped) — current state can always be derived by replaying those events. This gives a complete, immutable audit trail.

Java Code — Event-Sourced Order Aggregate

```
sealed interface OrderEvent permits OrderPlaced, PaymentCharged, OrderShipped, OrderCancelled
{} // Java 17 sealed (Book 07)
record OrderPlaced(Long orderId, Long customerId, BigDecimal total) implements OrderEvent {}
record PaymentCharged(Long orderId, BigDecimal amount) implements OrderEvent {}
record OrderShipped(Long orderId, String trackingNumber) implements OrderEvent {}
record OrderCancelled(Long orderId, String reason) implements OrderEvent {}

class Order { // rebuilt by replaying
    events, not read from one row
    private Long id;
    private OrderStatus status;
    private BigDecimal amountPaid = BigDecimal.ZERO;

    static Order replay(List<OrderEvent> events) {
        Order order = new Order();
        for (OrderEvent event : events) {
            order.apply(event); // Book 07 - pattern
        }
        return order;
    }

    private void apply(OrderEvent event) {
        switch (event) {
            case OrderPlaced e -> { this.id = e.orderId(); this.status = OrderStatus.PLACED; }
            case PaymentCharged e -> { this.amountPaid = this.amountPaid.add(e.amount()); }
            case OrderShipped e -> this.status = OrderStatus.SHIPPED;
            case OrderCancelled e -> this.status = OrderStatus.CANCELLED;
        }
    }
}
```

Internal Working

- The **event store** (a table of `(order_id, sequence_number, event_type, event_payload)` rows, append-only, never updated or deleted) is the true source of truth — the `Order` object above is a **projection**, rebuilt on demand by folding all events for that ID in sequence, using exactly Book 07's Java 17 sealed-type pattern-matching `switch` for exhaustive, type-safe event handling.
- For performance, systems periodically save a **snapshot** (the fully-computed `Order` state at event #500) so replay only needs to process events after the snapshot, not from event #1 every time.
- Event Sourcing pairs naturally with Ch.9's CQRS: the write side appends events; a projector (the same mechanism as Ch.9) builds the denormalized read model by consuming those events.

Real-World Example

Banking and accounting systems have used event-sourcing-like append-only ledgers for centuries (a bank statement is literally a sequence of transactions, not a single mutable balance) — modern event sourcing applies the same principle to any domain needing a full, immutable audit trail.

Interview Answer

"Event Sourcing stores the sequence of events that led to an entity's current state, rather than the state itself — the entity is a projection, rebuilt by replaying its events in order. This gives a complete, immutable audit trail and the ability to reconstruct state as of any point in time. It pairs naturally with CQRS: the write side appends events, and a read-model projector consumes them to build query-optimized views. The main cost is added complexity — replay logic, snapshotting for performance, and handling event schema evolution over time."

Cross Questions

- Q: What is actually stored as the source of truth in Event Sourcing? → A: The append-only sequence of events, not the entity's current state — current state is derived by replaying events.
- Q: Why would a system add snapshots to an event-sourced aggregate? → A: To avoid replaying every event from the beginning every time — a snapshot captures computed state at a point, so only events after it need replaying.

Tricky Questions

- Q: Is Event Sourcing the same thing as CQRS? → A: No — they're complementary but independent patterns; CQRS is about separating read/write models, Event Sourcing is about how the write side stores its data (as events, not current state). They're commonly used together but neither requires the other.

Coding Exercise

L1: Model an `Order` aggregate as a sealed event hierarchy with a `replay` method. **L2:** Add a new event type and update the `apply` switch to handle it. **L3:** Implement a simple snapshot mechanism (state at event #N). **L4 (Interview):** Explain Event Sourcing and how current state is derived. **L5**

(Senior): Design event schema evolution handling for a breaking event-shape change. **L6**
(Mastery): Design a full event-sourced + CQRS order system with snapshotting.

CHAPTER 11 — Distributed Tracing & Centralized Logging

Telugu Explanation

ఒక్క request Order → Payment → Notification services మూడింటినీ దాటితే, ఏదైనా fail అయితే ఏ service లో fail అయ్యిందో logs లో వెతకడం కష్టం, ఎందుకంటే ప్రతి service తన సొంత log file కి log చేస్తుంది. **Distributed Tracing** (Micrometer Tracing + Zipkin) ప్రతి request కి ఒక **trace ID** ఇచ్చి, ఆ ID ని అన్ని services మధ్య propagate చేస్తుంది; **Centralized Logging** (ELK stack) అన్ని services logs ని ఒక్క చోట aggregate చేస్తుంది.

Professional English Explanation

When a single request crosses Order → Payment → Notification services, finding where it failed is hard because each service logs to its own file. **Distributed Tracing** (Micrometer Tracing + Zipkin) assigns each request a **trace ID** and propagates it across all services; **Centralized Logging** (the ELK stack) aggregates all services' logs into one searchable place.

Java Code — Trace Propagation

```
// build.gradle: implementation 'io.micrometer:micrometer-tracing-bridge-brave'
//                  implementation 'io.zipkin.reporter2:zipkin-reporter-brave'

@RestController
class OrderController {
    private static final Logger log = LoggerFactory.getLogger(OrderController.class);

    @PostMapping("/api/orders")
    OrderResponse placeOrder(@RequestBody OrderRequest request) {
        log.info("Placing order for customer {}", request.customerId());
        // Micrometer Tracing auto-injects traceId/spanId into this log line via MDC -
        // e.g.: "2024-01-15 10:22:01 [traceId=4bf92f3577b34da6, spanId=00f067aa0ba902b7]"
        Placing order..."
        return orderService.place(request);    // the SAME traceId propagates into the
        Payment/Notification calls (Ch.3)
    }
}
```

```
management:
  tracing:
    sampling:
      probability: 1.0          # trace 100% of requests (lower in high-traffic prod,
e.g. 0.1)
    zipkin:
      tracing:
        endpoint: http://localhost:9411/api/v2/spans
```

Internal Working

- Micrometer Tracing automatically injects a `traceId` (constant across the whole request's journey through every service) and a `spanId` (unique per service hop) into the outgoing HTTP headers of Ch.3's `RestClient` calls — this is how Payment Service's logs for the *same request* get

the *same* `traceId` as Order Service's logs, even though they're separate JVM processes with separate log files.

- **MDC** (Mapped Diagnostic Context, an SLF4J feature) is how the `traceId` gets automatically included in every log line without manually passing it through every method signature — Book 04's exception logs and every other log statement automatically carry it.
- Once logs from all services are shipped to a centralized store (Elasticsearch, via Logstash/Filebeat — the "ELK stack"), searching for one `traceId` surfaces the complete cross-service journey of a single request in one query, which is otherwise nearly impossible to reconstruct by manually reading separate log files.

Real-World Example

When a customer reports "my order failed," a support engineer searches the centralized log store for that request's `traceId` and instantly sees its full path: Order Service accepted it, Payment Service's call to it timed out, Ch.7's circuit breaker fallback triggered — all without SSH-ing into three different servers' log files.

Interview Answer

"Distributed tracing assigns each request a `traceId` that's propagated across every service it touches, with a unique `spanId` per hop, using Micrometer Tracing and a backend like Zipkin. Combined with MDC, this `traceId` automatically appears in every log line, and centralized logging (the ELK stack) aggregates logs from every service into one searchable store — so debugging a cross-service failure means searching for one trace ID instead of manually correlating separate log files across services."

Cross Questions

- Q: How does Payment Service's log line get the same `traceId` as the Order Service request that called it? → A: The `traceId` is automatically injected into the outgoing HTTP headers of the `RestClient` call (Ch.3) and picked back up by the receiving service's tracing instrumentation.
- Q: What SLF4J mechanism lets `traceId` appear in log lines without changing every method signature? → A: MDC (Mapped Diagnostic Context) — it's set once per request and automatically included in the logging pattern for every subsequent log statement on that thread.

Tricky Questions

- Q: Should `sampling.probability` be `1.0` (trace everything) in a high-traffic production system? → A: Usually not — tracing every single request at high volume adds real overhead and storage cost; production systems typically sample a fraction (e.g., `0.1` = 10%) while still tracing 100% of errors.

Coding Exercise

L1: Add Micrometer Tracing + Zipkin to a two-service call chain and view the trace in Zipkin's UI. **L2:** Confirm the same `traceId` appears in both services' log output for one request. **L3:** Change the sampling probability and observe the effect on trace volume. **L4 (Interview):** Explain distributed

tracing and MDC's role in propagating trace context to logs. **L5 (Senior):** Design a centralized logging pipeline (ELK) for a 6-service system. **L6 (Mastery):** Debug a simulated cross-service failure using only a `traceId` search.

CHAPTER 12 — Docker for Microservices

Telugu Explanation

Book 12, Ch.14 లో ఒక్క Spring Boot app ని Docker image గా build చేసాము. Microservices లో ప్రతి service తన సొంత Docker image కలిగి ఉంటుంది, మరియు అన్ని services ని కలిపి run చేయడానికి **Docker Compose** వాడతారు — local development/testing లో మొత్తం multi-service system ని ఒకే command తో start చేయవచ్చు.

Professional English Explanation

Book 12, Ch.14 built a Docker image for one Spring Boot app. In microservices, each service has its own Docker image, and **Docker Compose** runs the whole set of services together — for local development/testing, the entire multi-service system can be started with a single command.

Java Code — Multi-Service Docker Compose

```
# docker-compose.yml - the whole microservices system, for local dev
version: "3.9"
services:
  eureka-server:
    build: ./eureka-server
    ports: ["8761:8761"]

  config-server:
    build: ./config-server
    ports: ["8888:8888"]
    depends_on: [eureka-server]

  order-service:
    build: ./order-service
    ports: ["8081:8081"]
    environment:
      EUREKA_URI: http://eureka-server:8761/eureka # container-to-container name resolution
      CONFIG_SERVER_URI: http://config-server:8888
    depends_on: [eureka-server, config-server, order-db]

  order-db:
    image: postgres:16
    environment:
      POSTGRES_DB: orderdb
      POSTGRES_PASSWORD: secret

  payment-service:
    build: ./payment-service
    ports: ["8082:8082"]
    depends_on: [eureka-server, config-server]

  gateway:
    build: ./gateway
    ports: ["8080:8080"]
    depends_on: [eureka-server, order-service, payment-service]
```

Internal Working

- Inside Docker Compose's network, services address each other **by service name** (`http://eureka-server:8761`) — Docker's internal DNS resolves `eureka-server` to that container's IP, which is why the `environment` block above uses service names, not `localhost` (a container's `localhost` refers to itself, not sibling containers).
- `depends_on` controls **startup order** but not **readiness** — `order-service` starting after `order-db` 's container starts doesn't guarantee PostgreSQL is actually ready to accept connections yet; production setups add healthchecks or retry-on-connect logic (Book 09's connection-pool retry behavior) to handle this gap.
- Each service still gets its own independently-built Docker image (Book 12, Ch.14's multi-stage build pattern) — Compose only orchestrates running multiple pre-built images together, it does not change how any single image is built.

Real-World Example

A team's local development setup often mirrors this exact `docker-compose.yml` structure so every developer can run the full multi-service system (Eureka, Config Server, Gateway, and all backend services) on their laptop with one `docker compose up`, instead of manually starting eight separate JVM processes.

Interview Answer

"Each microservice gets its own Docker image, built with Book 12's Dockerfile pattern. Docker Compose orchestrates running the whole set of services together for local development — services communicate using each other's Compose service names, resolved via Docker's internal DNS, not `localhost` or hardcoded IPs. `depends_on` only controls startup order, not actual readiness, so production-grade setups add explicit healthchecks or retry logic for dependencies like the database."

Cross Questions

- Q: Why do services in `docker-compose.yml` refer to each other by service name instead of `localhost` ? → A: Because each service runs in its own container with its own network namespace — `localhost` inside a container refers to that container itself, not sibling containers; Docker's internal DNS resolves service names to the correct container IPs.
- Q: Does `depends_on` guarantee a dependency is ready to accept connections? → A: No — only that its container has started; actual readiness (e.g., PostgreSQL accepting connections) requires healthchecks or retry logic.

Tricky Questions

- Q: If `order-service` 's Dockerfile is unchanged, does adding it to `docker-compose.yml` change how its image is built? → A: No — Compose only orchestrates running already-built images together; each image is still built independently exactly as Book 12, Ch.14 described.

Coding Exercise

L1: Write a `docker-compose.yml` for Eureka Server + two backend services + Gateway. **L2:** Add a PostgreSQL service and connect one backend service to it via service name. **L3:** Add a healthcheck to the database service and a `depends_on: condition: service_healthy` on the dependent service. **L4 (Interview):** Explain Docker Compose's role vs. individual service Dockerfiles. **L5 (Senior):** Diagnose a "connection refused" startup race condition and fix it. **L6 (Mastery):** Design the full Compose file for the Ch.14 mini project's 5-service system.

CHAPTER 13 — Kubernetes Fundamentals for Java Developers

Telugu Explanation

Docker Compose local development కి బాగుంటుంది, కానీ production లో వందల containers ని manage చేయడానికి, auto-scaling, self-healing, rolling updates కోసం **Kubernetes (K8s)** వాడతారు. ఒక Java developer గా తెలుసుకోవాల్సిన core concepts: **Pod** (ఒకటి లేదా ఎక్కువ containers ఒక్క unit గా), **Deployment** (desired replica count maintain చేయడం), **Service** (stable network endpoint), **ConfigMap/Secret** (Book 12's `application.yml` /credentials కి K8s-native equivalent).

Professional English Explanation

Docker Compose is fine for local development, but production needs **Kubernetes (K8s)** to manage hundreds of containers with auto-scaling, self-healing, and rolling updates. Core concepts every Java developer should know: **Pod** (one or more containers as a single unit), **Deployment** (maintains a desired replica count), **Service** (a stable network endpoint), **ConfigMap/Secret** (the K8s-native equivalent of Book 12's `application.yml` /credentials).

Java Code — Kubernetes Deployment + Service YAML

```
# order-service-deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3                                # K8s maintains exactly 3 running Pods
  selector:
    matchLabels: { app: order-service }
  template:
    metadata:
      labels: { app: order-service }
    spec:
      containers:
        - name: order-service
          image: myregistry/order-service:1.0.0  # the image built in Ch.12
          ports: [{ containerPort: 8081 }]
          envFrom:
            - configMapRef: { name: order-service-config }  # externalized config (Book 12,
Ch.8)
          readinessProbe:
            # is this Pod ready to receive traffic?
            httpGet: { path: /actuator/health, port: 8081 } # Book 12, Ch.10's Actuator
            initialDelaySeconds: 10
      ---
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector: { app: order-service }
  ports: [{ port: 8081, targetPort: 8081 }]
  type: ClusterIP                                # stable internal DNS name: order-service
```

Internal Working

- A **Deployment** continuously reconciles reality against the desired state: if one of the 3 Pods crashes, Kubernetes automatically starts a replacement — this is **self-healing**, a capability plain Docker Compose does not provide.
- The `readinessProbe` calling Book 12, Ch.10's `/actuator/health` endpoint is exactly how Kubernetes decides whether a Pod should receive traffic — this is the K8s-native version of the readiness gap Ch.12 noted `depends_on` couldn't solve; Kubernetes actively polls, not just checks at startup.
- A **Service** gives a stable DNS name (`order-service`) and load-balances across all matching Pods (matched by label, not by fixed IP, since Pods are ephemeral and get new IPs when recreated) — this plays the same conceptual role Eureka (Ch.5) plays outside Kubernetes; in fact, many K8s-native microservices deployments **drop Eureka entirely** and rely on K8s Services for discovery instead.

Real-World Example

A production Java Spring Boot microservices fleet running on Kubernetes typically doesn't need Eureka at all — Kubernetes' built-in Service discovery and its Deployment's self-healing/auto-scaling replace much of what Netflix OSS (Eureka, Ribbon) was originally built to solve outside a container-orchestration platform.

Interview Answer

"Kubernetes manages containerized applications in production with Pods (the smallest deployable unit), Deployments (maintaining a desired replica count with self-healing), Services (stable, label-based load-balanced endpoints), and ConfigMaps/Secrets (externalized configuration, playing the same role as Book 12's `application.yml`). A Deployment's readiness probe, often hitting the same `/actuator/health` endpoint from Book 12, determines whether a Pod receives traffic — Kubernetes actively polls this, giving self-healing and safe rolling updates that plain Docker Compose can't provide."

Cross Questions

- Q: What K8s capability does plain Docker Compose lack, that a Deployment provides? → A: Self-healing — a Deployment automatically replaces a crashed Pod to maintain the desired replica count; Compose does not do this by default.
- Q: Why might a Kubernetes-native microservices system not need Eureka? → A: Kubernetes Services already provide stable, label-based service discovery and load balancing, covering much of what Eureka was built to solve outside a container-orchestration platform.

Tricky Questions

- Q: Does a Kubernetes Service route traffic to a Pod by its IP address? → A: No — Pod IPs are ephemeral and change when Pods are recreated; a Service selects Pods dynamically by **label**, so it keeps routing correctly even as underlying Pods come and go.

Coding Exercise

L1: Write a Deployment YAML for a service with 3 replicas and a readiness probe. **L2:** Write a matching Service YAML exposing that Deployment. **L3:** Add a ConfigMap and reference it from the Deployment via `envFrom`. **L4 (Interview):** Explain Pods, Deployments, and Services and how they relate. **L5 (Senior):** Justify whether a K8s-native system still needs Eureka. **L6 (Mastery):** Design the full K8s manifests (Deployment + Service + ConfigMap) for the Ch.14 mini project.

CHAPTER 14 — Mini Project: Order + Payment + Notification Microservices System

Telugu Explanation

ఈ mini project లో ఇప్పటివరకు నేర్చుకున్న ప్రతిదాన్ని కలిపి — Config Server, Eureka, Gateway, Resilience4j, SAGA, Docker Compose — ఒక పూర్తి working multi-service system నా build చేస్తాము: Order Service ఒక order create చేస్తే, Payment Service ని sync నా call చేసి charge చేస్తుంది (Circuit Breaker తో protect చేయబడి), success అయితే Notification Service కి event పంపుతుంది.

Professional English Explanation

This mini project combines everything from this book — Config Server, Eureka, Gateway, Resilience4j, SAGA, Docker Compose — into one complete, working multi-service system: Order Service creates an order, synchronously calls Payment Service to charge it (protected by a Circuit Breaker), and on success, sends an event to Notification Service.

Project Structure

```
microservices-order-system/
├── eureka-server/           (Ch.5)
├── config-server/          (Ch.4)
├── gateway/                (Ch.6)
├── order-service/          (Ch.2, Ch.3, Ch.8 SAGA orchestrator)
├── payment-service/        (Ch.7 Circuit Breaker as the callee)
├── notification-service/   (Ch.3 async consumer, full Kafka version in Book 17)
└── docker-compose.yml      (Ch.12)
```

Java Code — Order Service Orchestrating the Full Flow

```
@RestController
@RequestMapping("/api/orders")
class OrderController {
    private final OrderSagaOrchestrator saga; // Ch.8

    @PostMapping
    ResponseEntity<OrderResponse> placeOrder(@Valid @RequestBody OrderRequest request) { //
Book 12, Ch.5 - validation
        Order order = saga.placeOrder(request); // Ch.3 sync call + Ch.7
resilience + Ch.8 compensation
        return ResponseEntity.status(HttpStatus.CREATED).body(OrderResponse.from(order));
    }

    @GetMapping("/{id}")
    OrderResponse getOrder(@PathVariable Long id) {
        return OrderResponse.from(orderService.findById(id)); // Book 13's JPA
repository
    }
}
```

Internal Working

- The full request path is: React app (Book 15A) → Gateway (Ch.6, JWT-validated per Book 14) → `lb://order-service` (Ch.5, Eureka-resolved) → `OrderSagaOrchestrator` (Ch.8) → `PaymentClient` (Ch.7, Circuit-Breaker-protected sync call to Payment Service) → on success, an event to Notification Service.
- Every service registers with Eureka (Ch.5), fetches config from Config Server (Ch.4), and is independently Dockerized (Ch.12) — this is the first chapter where all of this book's individually-taught pieces operate together as one coherent system, exactly the way Book 12's mini project synthesized that book's individual chapters.
- This same system, run on Kubernetes (Ch.13) instead of Docker Compose, would replace Eureka's registration with K8s Service discovery and replace manual scaling with `kubectl scale deployment order-service --replicas=5`.

Real-World Example

This mini project's shape — Gateway + Discovery + Config + resilient sync calls + compensating transactions — is the standard reference architecture for a mid-sized production Java microservices system, and is exactly the kind of system a "Java Full Stack Developer, 3-5 years" interview will ask candidates to design or debug.

Interview Answer

"This project demonstrates a complete microservices request flow: a client request enters through an API Gateway, is routed to a service discovered via Eureka, whose business logic orchestrates a SAGA across multiple services using resilient, circuit-breaker-protected synchronous calls, with compensating transactions on failure — all services centrally configured via Config Server and independently containerized. It's the standard shape recruiters are testing for when they ask a Java Full Stack Developer to 'design a microservices order system.'"

Coding Exercise

L1: Implement Order Service and Payment Service with Eureka registration and a sync call between them. **L2:** Add the Gateway routing both services behind one entry point. **L3:** Add Resilience4j Circuit Breaker to the Order → Payment call with a fallback. **L4 (Interview):** Walk through the full request path from client to Notification Service. **L5 (Senior):** Add Config Server and externalize all three services' configuration. **L6 (Mastery):** Containerize the full system with Docker Compose and verify it runs end-to-end with `docker compose up`.

FINAL REVISION NOTES

- Microservices trade a monolith's simplicity for independent scaling/deployment — justified by organizational/scaling needs, not fashion.
 - Service boundaries follow DDD's bounded contexts; reference other services by ID, snapshot data at a point in time when needed.
 - Sync (REST) communication couples availability; async (Kafka, Book 17) decouples it at the cost of eventual consistency.
 - Config Server centralizes configuration; Eureka centralizes service discovery; both are foundational to everything else in this book.
 - API Gateway is the single entry point, centralizing routing, rate limiting, and cross-cutting concerns.
 - Resilience4j's Circuit Breaker/Retry/Timeout/Bulkhead prevent cascading failure — the single most interview-tested microservices topic.
 - SAGA replaces distributed ACID transactions with local transactions + compensating actions.
 - CQRS and Event Sourcing are advanced, situational patterns — not defaults for every service.
 - Distributed tracing (`traceId` via Micrometer Tracing/Zipkin) and centralized logging (ELK) make cross-service debugging tractable.
 - Docker Compose is for local multi-service development; Kubernetes is the production-grade orchestration platform, and often replaces Eureka's role natively.
-



CHEAT SHEET

Concept	One-Line Summary
Bounded Context	One service = one business capability + its own data
<code>RestClient</code> (Ch.3)	Modern sync HTTP client, replaces <code>RestTemplate</code>
Config Server	Centralized, git-backed config, fetched via <code>spring.config.import</code>
Eureka	Service registry; <code>@LoadBalanced</code> + <code>lb://</code> resolve logical names
Spring Cloud Gateway	Single entry point; <code>lb://service-name</code> routes, filters add cross-cutting concerns
Circuit Breaker states	CLOSED → OPEN (fail fast) → HALF_OPEN (test) → CLOSED/OPEN
SAGA	Local transactions + compensating transactions; orchestration vs choreography
CQRS	Separate command (write) and query (read) models, synced via events
Event Sourcing	Store events, not state; state = replay of events
Docker Compose	Orchestrates multiple pre-built images for local dev; <code>depends_on</code> ≠ readiness
Kubernetes	Pod/Deployment/Service/ConfigMap; self-healing + native service discovery



INTERVIEW QUESTION BANK — Microservices

Beginner

1. What is a microservice, and how does it differ from a module in a monolith?
2. What is service discovery, and why is it needed?
3. What does an API Gateway do?

Intermediate 4. Explain the Circuit Breaker pattern and its three states. 5. What is the difference between synchronous and asynchronous inter-service communication? 6. How does Spring Cloud Config Server centralize configuration? 7. What problem does the SAGA pattern solve?

Advanced 8. Compare SAGA orchestration vs choreography. 9. Explain CQRS and when it's justified. 10. How does Event Sourcing derive current state? 11. Explain distributed tracing and how a `traceId` propagates across services.

Senior/Architect 12. Design a resilience strategy (timeout + retry + circuit breaker + bulkhead) for a critical payment call, addressing idempotency. 13. Design the full bounded-context decomposition for a ride-sharing platform. 14. Justify when a Kubernetes-native deployment can eliminate the need for Eureka. 15. Design a SAGA with CQRS and Event Sourcing combined for an auditable order system.



CROSS-QUESTION ENGINE — Sample Chains

- Q: Why does Eureka favor availability over consistency? → A: Stale-but-available service discovery keeps the system functioning; unavailability blocks all inter-service communication. → Cross: What handles a stale registry entry pointing to a now-dead instance? → A: The caller's Circuit Breaker/Retry (Ch.7) — a failed call to a dead instance is exactly the failure mode those patterns are designed to absorb.
 - Q: Why is 2PC avoided across microservice databases? → A: It requires all participating databases to support a distributed transaction coordinator and blocks on the slowest participant — impractical across independently-owned services. → Cross: What does SAGA use instead? → A: Local transactions plus explicit compensating transactions for eventual consistency.
-



CONSOLIDATED EXERCISES (All Levels)

- Design bounded contexts for a 5-service e-commerce platform.
 - Set up Eureka + Config Server + Gateway + two backend services end-to-end.
 - Implement full Resilience4j protection (Circuit Breaker + Retry + Timeout) on one inter-service call.
 - Implement a two-step SAGA with compensating transactions.
 - Containerize the full mini project with Docker Compose.
 - Write Kubernetes manifests for one service (Deployment + Service + ConfigMap).
-



ONE-DAY REVISION PLAN (≈6 hours)

Time	Focus
0:00–1:00	Ch.1–3: Monolith vs microservices, boundaries, sync/async
1:00–2:00	Ch.4–5: Config Server + Eureka
2:00–3:00	Ch.6–7: Gateway + Resilience4j (the two highest-weight chapters)
3:00–4:00	Ch.8–10: SAGA, CQRS, Event Sourcing
4:00–5:00	Ch.11–13: Tracing/logging, Docker, Kubernetes
5:00–6:00	Ch.14 mini project + full interview bank



ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1-3 — foundations and communication styles
2	Ch.4-5 — Config Server and Eureka, hands-on
3	Ch.6 — API Gateway, hands-on routing/rate limiting
4	Ch.7 — Resilience4j, hands-on all four patterns
5	Ch.8-10 — SAGA/CQRS/Event Sourcing
6	Ch.11-13 — tracing, logging, Docker, Kubernetes
7	Ch.14 mini project end-to-end + mock interview using the question bank



FINAL MASTERY CHECKLIST

- ☐ I can explain when microservices are justified vs. a monolith.
- ☐ I can design bounded contexts for a multi-service system.
- ☐ I can set up and explain Config Server and Eureka.
- ☐ I can configure an API Gateway with routing, rate limiting, and circuit breaking.
- ☐ I can implement and explain Circuit Breaker, Retry, Timeout, and Bulkhead with Resilience4j.
- ☐ I can design a SAGA (orchestration and choreography) with compensating transactions.
- ☐ I can explain CQRS and Event Sourcing and when each is justified.
- ☐ I can explain distributed tracing and centralized logging for cross-service debugging.
- ☐ I can containerize a multi-service system with Docker Compose.
- ☐ I can write basic Kubernetes manifests and explain Pod/Deployment/Service/ConfigMap.
- ☐ I completed the full mini project end-to-end.

Next: `17_Kafka_Event_Driven_Architecture.md` — Book 17, going deeper into the asynchronous communication this book referenced throughout (Ch.3, Ch.14).